
Deckblatt: Übungsblatt-Abgabe

Nummer des Übungsblattes: 5

Name Teammitglied 1: Jonas Dröse

Name Teammitglied 2: Sandra Gündel

Name Teammitglied 3: Philipp Hofer

Mit der Abgabe dieses Übungsblattes bestätigen wir, dass wir unsere Lösungen vollkommen **selbstständig** erstellt haben. Auf künstliche Intelligenz basierende Tools haben wir ausschließlich für die folgenden Aufgaben verwendet (Zutreffende bitte ankreuzen):

- ☒ Einfacheres Erklären von Vorlesungsinhalten / Klären von Verständnisfragen
- ☐ Hilfe beim Finden von Fehlern in der eigenen Lösung / Eigene Lösung überprüfen
- ☐ Umformulieren von Sätzen für leichtere Lesbarkeit
- ☐ Übersetzen von Aufgabenstellungen und Erklärungen in eine andere Sprache
- ☐ Generieren von unterstützenden Abbildungen oder Tabellen
- ☒ Ideenfindung und Finden von Lösungsansätzen
- ☐ Generieren von zusätzlichen Übungsaufgaben zu Vorlesungsinhalten
- ☒ Recherchieren zu Vorlesungsthemen
- ☐ Überprüfen von Rechtschreibung und Grammatik

Insbesondere haben wir auf künstliche Intelligenz basierende Tools **nicht** genutzt, um die Aufgaben lösen zu lassen. Sämtliche von künstlicher Intelligenz generierten Inhalte haben wir kritisch geprüft und hinterfragt. Wir bestätigen außerdem, dass wir das geistige Eigentum der Autoren der Vorlesungsmaterialien gewahrt haben, indem wir **keine** PDF-Dateien des Übungsblatts oder von Kapiteln des Skripts (auch in Auszügen) auf externen Servern (z.B. bei ChatGPT) hochgeladen haben. Uns ist bewusst, dass eine Falschangabe prüfungsrechtliche Konsequenzen für uns nach sich ziehen kann.

Aufgabe 17:

Begriffe bei indexOf(Object o)

Substitutionsprinzip: Da der Parameter als Object deklariert ist, kann jede beliebige Klasse übergeben werden. Jede Unterklasse ist anstelle ihrer Oberklasse einsetzbar, da alle Object die Oberklasse von allen Klassen ist.

Polymorphismus: Die Fähigkeit des Objekts o, zur Laufzeit unterschiedliche Formen anzunehmen. Obwohl der statische Typ Object ist, agiert o als Instanz der tatsächlich übergebenen Klasse.

Spätes Binden: Erst zur Laufzeit wird entschieden, welche konkrete equals()-Methode aufgerufen wird (z. B. die von String oder Integer). Die JVM prüft den tatsächlichen Typ des Objekts auf dem Heap und nicht die Referenzvariable.

Vorteile von Interfaces bei addAll(Collection<? extends E> c)

Interoperabilität: Die Methode ist nicht auf ArrayList beschränkt. Sie kann Daten aus jeder Klasse verarbeiten, die Collection implementiert (z. B. HashSet, LinkedList).

Abstraktion: Die Implementierung muss nur den "Vertrag" des Interfaces kennen (z. B. Iterierbarkeit), nicht die interne Speicherstruktur der übergebenen Sammlung.

Wartbarkeit: Änderungen an der konkreten Implementierung der übergebenen Collection haben keinen Einfluss auf die addAll-Methode, was den Code entkoppelt und flexibel hält.

Aufgabe 18:

```
package Übungsblatt5;
```

```
import java.util.ArrayList;
```

```
import java.util.ListIterator;
```

```
public class Aufgabe18a {
```

```
    public static void main(String[] args) {
```

```
        ArrayList<String> myList = new ArrayList<>();
```

```
        myList.add("Mein");
```

```
        myList.add("kleiner");
```

```
myList.add("grüner");
```

```
for (int i = 0; i < args.length; i++) {  
    myList.add(3, args[i]);  
}
```

```
if (!myList.contains("Hollari")) {  
    myList.add("Hollari");  
}
```

```
if (!myList.contains("hollari")) {  
    myList.add("hollari");  
}
```

```
if (!myList.contains("hollaro")) {  
    myList.add("hollaro");  
}
```

```
ListIterator<String> iter = myList.listIterator(myList.size() - 5);
```

```
while (iter.hasNext()) {  
    System.out.print(iter.next());  
  
    if (iter.hasNext()) {  
        System.out.print(", ");  
    }  
  
    if (!iter.hasNext()) {  
        System.out.print("!");  
    }  
}
```

```
    }  
}
```

```
package Übungsblatt5;
```

```
import java.util.HashSet;
```

```
import java.util.ArrayList;
```

```
public class Aufgabe18b {
```

```
    public static void main(String[] args) {
```

```
        HashSet<Character> mySet = new HashSet<>();
```

```
        for (int i = 0; i < args.length; i++) {
```

```
            char[] zeichenArray = args[i].toCharArray();
```

```
            for (int j = 0; j < zeichenArray.length; j++) {
```

```
                mySet.add(zeichenArray[j]);
```

```
            }
```

```
        }
```

```
        System.out.println(mySet);
```

```
        for (Character zeichen : mySet) {
```

```
            System.out.println(zeichen);
```

```

        ArrayList<String> sammelbox = new ArrayList<>();
        for (int i = 0; i < args.length; i++) {
            if (args[i].contains(String.valueOf(zeichen))){
                sammelbox.add(args[i]);
            }
        }

        System.out.println(String.join(" ", sammelbox));
    }
}

```

```

package Übungsblatt5;

```

```

import java.util.HashSet;
import java.util.ArrayList;
import java.util.Arrays;

```

```

public class Aufgabe18c {

```

```

    public static void main(String[] args) {

```

```

        HashSet<Character> mySet = new HashSet<>();

```

```

        for (String arg : args) {
            for (char c : arg.toCharArray()) {
                mySet.add(c);
            }
        }

```

```

        Object[] myArray = mySet.toArray();

```

```
System.out.println(Arrays.toString(myArray));
```

```
for (Object obj : myArray) {
```

```
    Character zeichen = (Character) obj;
```

```
    System.out.println(zeichen);
```

```
    ArrayList<String> sammelbox = new ArrayList<>();
```

```
    for (String arg : args) {
```

```
        if (arg.contains(String.valueOf(zeichen))) {
```

```
            sammelbox.add(arg);
```

```
        }
```

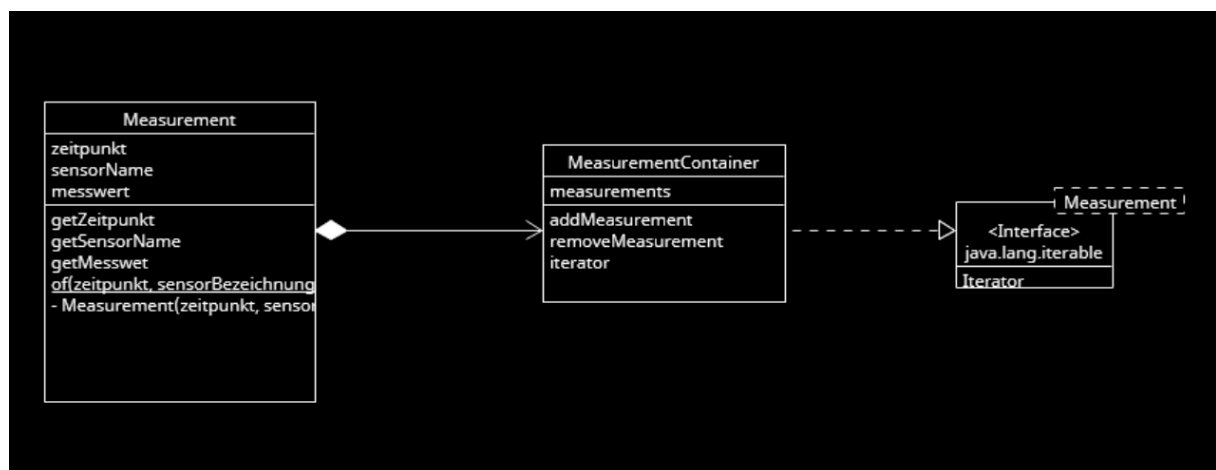
```
    }
```

```
    System.out.println(String.join(" ", sammelbox));
```

```
}
```

```
}
```

Aufgabe 19



```
package Übungsblatt5;
```

```
import java.time.*;
```

```
public final class Measurement19b {
```

```

private final LocalDateTime zeitpunkt;

private final String sensorBezeichnung;

private final double messwert;

private Measurement19b (LocalDateTime zeitpunkt, String sensorBezeichnung, double
messwert) {

    this.zeitpunkt = zeitpunkt;

    this.sensorBezeichnung = sensorBezeichnung;

    this.messwert = messwert;

}

public LocalDateTime getZeipunkt() {

    return zeitpunkt;

}

public String getSensorBezeichnung() {

    return sensorBezeichnung;

}

public double getMesswert() {

    return messwert;

}

public static Measurement19b of (LocalDateTime zeitpunkt, String sensorBezeichnung,
double messwert) {

    return new Measurement19b(zeitpunkt, sensorBezeichnung, messwert);

}

}

package Übungsblatt5;

```

```
import java.util.ArrayList;

import java.util.Iterator;


public class MeasurementContainer19c implements Iterable<Measurement19b> {


    private final ArrayList<Measurement19b> measurements = new
ArrayList<Measurement19b>();


    public void addMeasurement(Measurement19b m) {
        measurements.add(m);
    }


    public void removeMeasurement(Measurement19b m) {
        measurements.remove(m);
    }


    @Override
    public Iterator<Measurement19b> iterator() {
        return measurements.iterator();
    }

}
```

```
package Übungsblatt5;
```

```
import java.time.LocalDateTime;
import java.util.Scanner;
```

```
public class MeasurementTest {

    public static void main(String[] args) {
```

```
Scanner scanner = new Scanner(System.in);
```

```
System.out.print("Sensor name: ");
```

```
String sensor = scanner.nextLine();
```

```
MeasurementContainer19c container = new MeasurementContainer19c();
```

```
System.out.println("Enter measurements (any non-numeric input will stop the program):");
```

```
while (scanner.hasNextDouble()) {
```

```
    double value = scanner.nextDouble();
```

```
    Measurement19b m = Measurement19b.of(LocalDate.now(), sensor, value);
```

```
    container.addMeasurement(m);
```

```
}
```

```
double sum = 0.0;
```

```
int count = 0;
```

```
for (Measurement19b m : container) {
```

```
    sum = sum + m.getMesswert();
```

```
    count++;
```

```
}
```

```
if (count == 0) {
```

```
    System.out.println("No measurements registered.");
```

```
    return;
```

```
}
```

```
double mean = sum / count;
```

```
System.out.println("Mean of " + count + " Measurements: " + mean);
```

```
}
```

```
}
```

```
package Übungsblatt5;
```

```
import java.util.ArrayList;
```

```
import java.util.Arrays;
```

```
import java.util.function.*;
```

```
public class Aufgabe20a {
```

```
    public static void main(String[] args) {
```

```
        ArrayList<String> parameter = new ArrayList<>(Arrays.asList(args));
```

```
        System.out.println("Übergebene Parameter" + parameter);
```

```
        UnaryOperator<String> upper = s -> {return s.toUpperCase();};
```

```
        parameter.replaceAll(upper);
```

```
        parameter.removeIf (a -> a.startsWith("A"));
```

```
        parameter.forEach(System.out::println);
```

```
    }
```

```
}
```

```
package Übungsblatt5;
```

```
import java.util.Random;
```

```
import java.util.ArrayList;

public class Aufgabe20b {

    public static void main(String[] args) {

        ArrayList<Integer> myList = new ArrayList<Integer>();
        Random random = new Random();

        for (int i = 0; i < 5000; i++) {
            myList.add(random.nextInt(10000));
        }

        myList.replaceAll(a -> (a % 2 == 0) ? a * a : a);

        myList.removeIf(n -> {
            if (n < 2) return false;
            for (int i = 2; i <= Math.sqrt(n); i++) {
                if (n % i == 0) {
                    return false;
                }
            }
        });

        myList.forEach(System.out::println);
        System.out.println("Anzahl Zahlen:" +myList.size());

    }
```

}